

Python Programming

Unit 02 – Lecture 03 Notes

Sets and Dictionaries

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	2
2	Part A: Sets	2
2.1	What Is a Set? (Definition and Properties)	2
2.2	Sets vs Lists vs Tuples (When to Use What)	2
2.3	Set Initialization (Including Empty Sets)	3
2.4	Hashability Rule (Why Lists Cannot Be in Sets)	3
2.5	Core Set Methods	3
2.6	Set Operations (Operators and Methods)	4
2.7	Set Comparisons (Subset, Superset, Disjoint)	4
2.8	Mini Use Cases for Sets	4
3	Part B: Dictionaries	5
3.1	What Is a Dictionary? (Definition and Key Rules)	5
3.2	Ways to Create Dictionaries	5
3.3	Essential Dictionary Views: <code>keys</code> , <code>values</code> , <code>items</code>	5
3.4	Safe Access: <code>d[key]</code> vs <code>d.get(key)</code>	5
3.5	Updating and Removing Data	6
3.6	Classic Pattern: Frequency Counting	6
3.7	Nested Dictionaries and Lists (JSON-like Data)	6
3.8	Sorting and Ranking with Dictionaries	6
4	Typecasting and Converting Between Collections	7
4.1	Common Conversions	7
4.2	Why Convert? (Typical Reasons)	7
5	Demo Walkthrough: Word Frequency and Unique Words	7
5.1	What the Demo Teaches	7
5.2	Suggested Run Steps	7

6 Quiz and Solutions	7
6.1 Checkpoint 1 (Empty Set)	8
6.2 Checkpoint 2 (remove vs discard)	8
6.3 Checkpoint 3 (Nested Access)	8
6.4 Additional Knowledge Checks (With Answers)	8
7 Practice Exercises (With Solutions)	8
7.1 Exercise 1: Remove Duplicate Emails	8
7.2 Exercise 2: Common Students in Two Clubs	9
7.3 Exercise 3: Shopping Cart (Dictionary Case Study)	9
7.4 Exercise 4: Sort a Dictionary by Values	9
7.5 Exercise 5: Nested Dictionary Safe Access	9
8 Mini-Project Ideas (Optional)	10
9 Further Reading	10

1 Lecture Overview

This lecture covers two high-impact Python collections:

- **Sets** (`set`): best for *uniqueness* and *membership checks*.
- **Dictionaries** (`dict`): best for *key-value modeling*, *fast lookup*, and *counting*.

If you remember only one idea:

Use a **set** when you care about “is it present?” and uniqueness.

Use a **dictionary** when you care about “for this key, what value should I store?”

2 Part A: Sets

2.1 What Is a Set? (Definition and Properties)

A **set** is an **unordered** collection of **unique** elements. Key properties:

- **Unique:** duplicates are automatically removed.
- **Unordered:** there is no stable index-based position.
- **Mutable:** you can add and remove elements.
- **Hashable-only elements:** elements must be immutable/hashable (e.g., `int`, `str`, `tuple`).

2.2 Sets vs Lists vs Tuples (When to Use What)

Feature	List	Tuple	Set
Order matters	Yes	Yes	No
Allows duplicates	Yes	Yes	No
Mutable container	Yes	No	Yes
Fast membership (<code>in</code>)	Slow for big lists	Slow	Fast

Practical decision:

- If you need order (and indexing/slicing), use a **list** or **tuple**.
- If you need uniqueness and fast membership, use a **set**.
- If values should not change (fixed record), use a **tuple**.

2.3 Set Initialization (Including Empty Sets)

Common ways to create sets:

```
A = {1, 2, 2, 3} # duplicates removed -> {1, 2, 3}
B = set([3, 4, 4, 5]) # from an iterable
C = set("mississippi") # from a string
empty = set() # empty set
```

Important: {} creates an *empty dictionary*, not an empty set.

2.4 Hashability Rule (Why Lists Cannot Be in Sets)

Because a set uses hashing internally, elements must be hashable:

```
ok = {(1, 2), (3, 4)} # tuples are hashable if elements are hashable
# bad = {[1, 2], [3, 4]} # ERROR: list is not hashable
```

2.5 Core Set Methods

Sets support a small set of methods that cover most real use cases.

Adding Elements: add and update

```
s = {1, 2}
s.add(3) # add one element
s.update([4, 5, 5]) # add many (duplicates ignored)
```

Removing Elements: remove vs discard

```
s = {1, 2, 3}
s.remove(2) # removes 2
# s.remove(99) # KeyError if missing
s.discard(99) # safe: no error if missing
```

Rule of thumb:

- Use **remove** when missing items are a true error.
- Use **discard** when missing items are expected.

Other Useful Methods: pop, clear, copy

```
s = {10, 20, 30}
item = s.pop() # removes an arbitrary element (not "first")
s.clear() # empties the set
t = s.copy() # copy of a set
```

2.6 Set Operations (Operators and Methods)

Set operations are used to compare groups of items. Let A and B be sets:

- **Union:** elements in either set $A \mid B$
- **Intersection:** elements common to both $A \& B$
- **Difference:** elements in A but not in B $A - B$
- **Symmetric difference:** elements in exactly one set $A \wedge B$

Example:

```
A = {1, 2, 3}
B = {2, 3, 4}
print(A | B) # {1, 2, 3, 4}
print(A & B) # {2, 3}
print(A - B) # {1}
print(A ^ B) # {1, 4}
```

Each operation also has a method form such as `A.union(B)`.

2.7 Set Comparisons (Subset, Superset, Disjoint)

Sometimes you want logical comparisons:

```
A = {1, 2}
B = {1, 2, 3}

print(A <= B) # True (A is subset of B)
print(B >= A) # True (B is superset of A)
print(A.isdisjoint({9})) # True (no common elements)
```

2.8 Mini Use Cases for Sets

- **Remove duplicates:** unique IDs/emails.
- **Seen-before logic:** track visited items.
- **Compare groups:** students in two clubs, common tags, etc.

3 Part B: Dictionaries

3.1 What Is a Dictionary? (Definition and Key Rules)

A **dictionary** stores **key–value pairs**. You access values by key, not by position:

```
student = {"name": "Asha", "marks": 88}
print(student["name"])
```

Key rules:

- Keys must be **unique** (a duplicate key overwrites the old value).
- Keys must be **hashable** (immutable), e.g., `str`, `int`, `tuple`.
- Values can be any type (including lists/dicts for nested structures).

Ordering note: In modern Python (3.7+), dictionaries preserve insertion order. Conceptually, still treat a dictionary as a *mapping* rather than a positional sequence.

3.2 Ways to Create Dictionaries

```
d1 = {"a": 1, "b": 2}
d2 = dict(a=1, b=2)
d3 = dict([("a", 1), ("b", 2)]) # list of tuples (pairs)
d4 = {} # empty dictionary
```

3.3 Essential Dictionary Views: keys, values, items

```
d = {"a": 1, "b": 2}
print(d.keys()) # keys view
print(d.values()) # values view
print(d.items()) # (key, value) pairs
```

These are *views*, not lists. Convert if you need a list:

```
keys_list = list(d.keys())
```

3.4 Safe Access: `d[key]` vs `d.get(key)`

`d[key]` raises `KeyError` if the key is missing. `d.get(key, default)` avoids that by returning a default.

```
d = {"a": 1}
# d["b"] # KeyError
print(d.get("b")) # None
print(d.get("b", 0)) # 0
```

Rule of thumb:

- Use `d[key]` when missing keys indicate a bug.
- Use `get` when missing keys are expected.

3.5 Updating and Removing Data

update for merging

```
d = {"a": 1, "b": 2}
d.update({"b": 3, "c": 4})
print(d) # {"a": 1, "b": 3, "c": 4}
```

pop and clear

```
d = {"a": 1, "b": 2}
val = d.pop("a") # removes key "a" and returns its value
d.clear() # empties the dict
```

3.6 Classic Pattern: Frequency Counting

Counting occurrences is one of the most common dictionary patterns.

```
text = "to be or not to be"
freq = {}
for w in text.split():
    freq[w] = freq.get(w, 0) + 1
```

The key idea: `get(w, 0)` gives 0 for unseen keys.

3.7 Nested Dictionaries and Lists (JSON-like Data)

Nested structures appear everywhere: API responses, configuration files, records.

```
students = {
    101: {"name": "Anu", "marks": 85},
    102: {"name": "Raj", "marks": 91},
}
print(students[101]["marks"]) # 85
```

List of dictionaries is also common:

```
rows = [
    {"name": "Asha", "cgpa": 9.1},
    {"name": "Bilal", "cgpa": 8.3},
]
top = max(rows, key=lambda r: r["cgpa"])
```

3.8 Sorting and Ranking with Dictionaries

Sorting a dictionary usually means sorting its `items()` (pairs).

```
counts = {"a": 3, "b": 1, "c": 2}
by_key = sorted(counts.items()) # sorts by key
by_value_desc = sorted(counts.items(), key=lambda kv: kv[1], reverse=True)
top2 = by_value_desc[:2]
```

Note: `sorted(...)` returns a *list*. This is often exactly what you want for reporting.

4 Typecasting and Converting Between Collections

Converting collection types is common in real work (data cleaning, compatibility, restructuring).

4.1 Common Conversions

```
L = [1, 2, 2, 3]
S = set(L) # list -> set (unique)
L2 = list(S) # set -> list (order may change)

T = (10, 20, 30)
L3 = list(T) # tuple -> list (to modify)
T2 = tuple(L3) # list -> tuple

pairs = [("a", 1), ("b", 2)]
d = dict(pairs) # list of tuples -> dict
```

4.2 Why Convert? (Typical Reasons)

- **Remove duplicates:** list $\rightarrow$ set $\rightarrow$ list.
- **Enable updates:** tuple $\rightarrow$ list (edit) $\rightarrow$ tuple.
- **Prepare inputs:** some APIs expect a specific type.
- **Data restructuring:** pairs $\rightarrow$ dictionary for fast lookup.

5 Demo Walkthrough: Word Frequency and Unique Words

Script: demo/word_frequency.py

5.1 What the Demo Teaches

- Use a set for unique vocabulary (de-duplication).
- Use a dictionary for counting occurrences.
- Sort results to generate a “top-k” list.

5.2 Suggested Run Steps

```
python demo/word_frequency.py
```

In class: ask students to *predict* the frequency dictionary before running.

6 Quiz and Solutions

This section includes slide checkpoints plus additional quick checks.

6.1 Checkpoint 1 (Empty Set)

Question. Which code correctly creates an empty set: `s = {}` or `s = set()`?

Solution. `s = set()` creates an empty set. `{}` creates an empty dictionary.

6.2 Checkpoint 2 (remove vs discard)

Question. What is the difference between `s.remove(x)` and `s.discard(x)`?

Solution. Both remove `x` if present. If `x` is missing, `remove` raises `KeyError` but `discard` does nothing.

6.3 Checkpoint 3 (Nested Access)

Question. Given:

```
response = {"user": {"profile": {"email": "k@gmail.com"}}
```

How do you access the email value?

Solution.

```
email = response["user"]["profile"]["email"]
```

6.4 Additional Knowledge Checks (With Answers)

Q1. Which operation returns elements present in either set, but not both?

A. Symmetric difference: `A ^ B`.

Q2. Which method adds multiple elements to a set at once?

A. `update(...)`.

Q3. Which method safely retrieves a value from a dictionary without raising an error for missing keys?

A. `get(key, default)`.

7 Practice Exercises (With Solutions)

These exercises mirror real tasks: data cleaning, membership checks, counting, and reporting.

7.1 Exercise 1: Remove Duplicate Emails

Task. Given a list of emails, remove duplicates.

Solution (order not guaranteed).

```
emails = ["a@x.com", "b@x.com", "a@x.com"]
unique = list(set(emails))
```

Better (preserve first-occurrence order).

```
def unique_keep_order(items):
    seen = set()
    out = []
    for x in items:
        if x not in seen:
            seen.add(x)
            out.append(x)
```



```
        out.append(x)
    return out
```

7.2 Exercise 2: Common Students in Two Clubs

Task. Given two sets of roll numbers, print students in both clubs.

Solution.

```
club_a = {101, 102, 103}
club_b = {102, 104}
common = club_a & club_b
print(common) # {102}
```

7.3 Exercise 3: Shopping Cart (Dictionary Case Study)

Task. Model a shopping cart as a dictionary item -> quantity. Implement add, remove, and total items.

Solution.

```
def add_item(cart, item, qty=1):
    cart[item] = cart.get(item, 0) + qty

def remove_item(cart, item):
    cart.pop(item, None) # safe removal

def total_items(cart):
    return sum(cart.values())

cart = {}
add_item(cart, "pen", 2)
add_item(cart, "book", 1)
remove_item(cart, "pen")
print(cart, total_items(cart))
```

7.4 Exercise 4: Sort a Dictionary by Values

Task. Sort a dictionary by value descending and print top 3 pairs.

Solution.

```
d = {"a": 3, "b": 1, "c": 2, "d": 5}
top3 = sorted(d.items(), key=lambda kv: kv[1], reverse=True)[:3]
print(top3)
```

7.5 Exercise 5: Nested Dictionary Safe Access

Task. Given nested dict `students`, safely read marks of roll 101.

Solution.

```
students = {101: {"name": "Anu", "marks": 85}}
marks = students.get(101, {}).get("marks")
print(marks) # 85
```

8 Mini-Project Ideas (Optional)

- **Student Record Manager:** store student profiles in a nested dictionary, compute averages, find toppers.
- **Email Deduplication Tool:** read emails, remove duplicates, report unique counts (set + list).
- **Library Inventory:** map book title to available copies; update on issue/return.
- **Shopping Cart Simulator:** add/remove items, compute totals, and generate receipts.

9 Further Reading

- <https://docs.python.org/3/tutorial/datastructures.html#sets>
- <https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Overview
- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Explain why sets guarantee uniqueness and why they do not support indexing
- Apply set methods (`add/update/remove/discard`) and set operations (union, intersection, etc.)
- Build dictionaries for real-world key–value modeling and safe lookup
- Work with nested dictionaries/lists and sort dictionary items by key/value
- Use typecasting (`list/tuple/set/dict`) for data cleaning and restructuring

Sets: Definition and Properties

- Unordered collection of **unique** elements
- Fast membership testing: `x in s`
- Mutable (you can add/remove elements)
- Elements must be **hashable** (e.g., `int/str/tuple`, not `list`)

Creating Sets (Including Empty Sets)

- Use braces for non-empty sets: `{1, 2, 3}`
- Use `set(...)` to convert from any iterable
- `{}` is an empty dictionary, so use `set()` for an empty set

```
A = {1, 2, 2, 3} # duplicates removed
B = set([3, 4, 4, 5]) # from list
empty = set() # NOT {}
```

Set Methods (Add/Remove/Clear/Copy)

Method	What it does
<code>add(x)</code>	add one element
<code>update(iter)</code>	add many elements
<code>remove(x)</code>	remove; error if missing
<code>discard(x)</code>	remove; no error if missing
<code>pop()</code>	remove and return an arbitrary element
<code>clear()</code>	remove all elements
<code>copy()</code>	shallow copy

Set Operations: Operators vs Methods

- Union: `A | B` or `A.union(B)`
- Intersection: `A & B` or `A.intersection(B)`
- Difference: `A - B` or `A.difference(B)`
- Symmetric difference: `A ^ B` or `A.symmetric_difference(B)`

```
A = {1, 2, 3}
B = {2, 3, 4}
print(A & B) # {2, 3}
```

Set Comparisons (Subset/Superset/Disjoint)

- Subset: `A <= B` or `A.issubset(B)`
- Superset: `A >= B` or `A.issuperset(B)`
- Disjoint: `A.isdisjoint(B)` (no common elements)

Dictionaries: Key–Value Model

- Map keys to values (fast lookup by key)
- Keys must be unique and hashable (immutable)
- Values can be any type (including lists/dicts)
- In modern Python, dictionaries preserve insertion order (but think “mapping”)

```
student = {"name": "Asha", "marks": 88}
print(student["name"])
```

Creating Dictionaries (Common Ways)

- Curly braces: `{"a": 1, "b": 2}`
- `dict()` constructor: `dict(a=1, b=2)`
- From a list of pairs: `dict([("a", 1), ("b", 2)])`

Essential Dictionary Methods

- Views: `d.keys()`, `d.values()`, `d.items()`
- Safe access: `d.get(key, default)`
- Update/merge: `d.update(other)`
- Removal: `d.pop(key)`, `d.popitem()`
- Cleanup: `d.clear()`

```
freq = {}  
word = "python"  
freq[word] = freq.get(word, 0) + 1
```

Nested Dictionaries and Lists

- Real data is often nested (JSON-like)
- Access step-by-step using keys and indices

```
response = {"user": {"profile": {"email": "k@gmail.com"}}}  
email = response["user"]["profile"]["email"]
```

Sorting Dictionary Data

- Sort by key: `sorted(d.items())`
- Sort by value: `sorted(d.items(), key=lambda kv: kv[1])`
- Ranking (top-k): sort then slice

```
counts = {"a": 3, "b": 1, "c": 2}  
top = sorted(counts.items(), key=lambda kv: kv[1], reverse=True)[:2]
```

Typecasting Between Collections (Data Cleaning)

- Remove duplicates: `list(set(data))` (order may change)
- Modify an immutable tuple: `tuple -> list -> tuple`
- Build dict from pairs: `dict([("a", 1), ("b", 2)])`

Where Sets and Dictionaries Shine

- Sets: unique email addresses, fast membership checks, comparing groups
- Dictionaries: student records, inventory, shopping carts, frequency counts
- Both: building reliable, structured data models for programs

Demo: Word Frequency and Unique Words

- Split a sentence into words
- Build a frequency dictionary (`get(..., 0)` pattern)
- Build a set of unique words
- Extension: print top 3 most frequent words

Script: `demo/word_frequency.py`

Checkpoint 1

Which code correctly creates an empty set?

1. `s = {}`
2. `s = set()`

Checkpoint 2

What is the difference between `s.remove(x)` and `s.discard(x)`?

Checkpoint 3

Given `response = {"user": {"profile": {"email": "k@gmail.com"}}}`,
write the expression to access the email value.

Think-Pair-Share

Choose the right collection type (list/tuple/set/dict) for:

- unique roll numbers in a class
- a fixed (x, y) coordinate
- a student's profile (name, roll, marks)

Summary

- Sets store unique, unordered values and support set algebra
- Dictionaries map keys to values for fast lookup and counting
- Nested structures are common in real data (JSON-like)
- Sorting and typecasting help with analytics and data cleaning

Exit Question

Write one line to sort dictionary items by value in descending order.